A portable `std::simd` FFT for GNU Radio 4

Dennis Klein
GSI Helmholtz Centre for Heavy Ion Research
d.klein@gsi.de

Claude
Anthropic

October 2026

Abstract

GNU Radio 4 computes FFTs with `SimdFFT`, a port of PFFFT whose vectors are fixed at four lanes. We present `fastfft`, a single-header Stockham FFT written against `std::simd` that uses the full vector width of the target and contains no intrinsics. On two AVX-512 machines it is 1.2–3.4× faster than `SimdFFT` for complex floats and roughly on par with FFTW. With C++26 `std::simd` in GCC trunk it is pure standard C++. Code, data and the measurement harness are public.

1 Introduction

GNU Radio 4 (gr4) [1] is a header-only C++23 rewrite of the GNU Radio framework. Its FFTs go through `SimdFFT`, a port of PFFFT [2] to `std::experimental::simd` [3] via vir-simd [5], which always uses vectors of four elements. On an AVX-512 machine that leaves three quarters of each register unused for floats. We asked how fast an FFT can be when written only against the native vector type, so that the width follows the target, and whether it can keep up with a tuned library.

2 Algorithm

`fastfft` is a Stockham autosort FFT [7, 8]: each pass reads one buffer and writes the other in natural order, so no bit-reversal is needed. It has three kinds of passes. The first, radix 8, works on several butterflies at once and transposes their results in registers before storing them. The middle passes, radix 16, work on independent sub-transforms that share a twiddle factor, so the twiddle is a single broadcast. A final radix 2, 4 or 8 pass handles what remains. Large radices mean few passes over memory: six instead of ten for a million points. Twiddle tables are laid out so that a complex product costs one multiply, one fused multiply-add and one in-register swap.

3 What `std::simd` could not express

`std::experimental::simd` has no operation that takes lanes from two vectors, which the transpose in the first pass needs; we use the compiler's generic vector extension there, still without intrinsics. Its `fma` is not inlined, so we write `a*b+c` and let the compiler contract it. With libc++, vir-simd silently emulates the vectors with arrays; the header reports this through `FASTFFT_NATIVE_SIMD`. C++26 `std::simd` [4], as implemented in GCC trunk, has the missing two-input permute, so with `-std=c++26` the header is pure standard C++. It also offers vectors of complex numbers, which we tried as well.

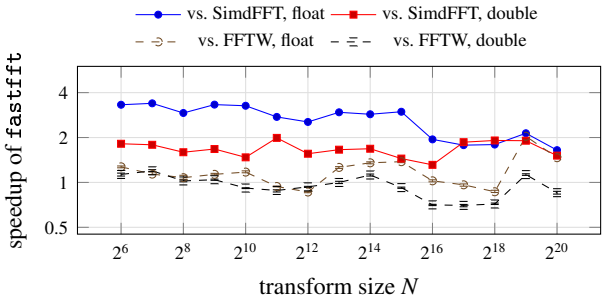


Figure 1: Speedup of `fastfft` (median of paired ratios, 95 % CI as error bars; mostly smaller than the markers). Above 1 means `fastfft` is faster. Absolute times in Tables 1 and 2.

4 Method

A benchmark program checks every implementation against a long-double reference DFT and then times it on a stream of frames, the way a gr4 block sees data. Each process measures all implementations in 40 rounds, in a new random order each round, pinned to one core; we run three processes. We report medians with 95 % bootstrap confidence intervals, and speedups as medians of ratios taken within the same round [9, 10]. Competitors are `SimdFFT` and FFTW 3.3.10 [6] built with AVX-512 support. Two hosts were measured: a virtual machine on an Intel Xeon (GCC 14.2) and a laptop with an AMD Ryzen AI 9 HX 370 (Zen 5, GCC 16.2.1), both with AVX-512 and both also with GCC trunk. The spread was small (relative median absolute deviation at most 10.2 %).

5 Results

Kernel. `fastfft` is faster than `SimdFFT` at every size on both hosts (Figure 1): 1.6–3.4× for floats on the VM and 1.2–2.7× on the laptop. The gain is largest for small and medium sizes and shrinks for large ones, where memory traffic dominates, and in double precision, where four lanes already fill half of a 512-bit register. Against FFTW the lead changes hands with size and precision (0.8–2.4× for floats, 0.7–1.5× for doubles); for floats `fastfft` wins most sizes from 2^{13} up. Accuracy matches both (Table 7).

Vector width. Compiled for 128-bit vectors, the width `SimdFFT` uses, `fastfft` is at most moderately faster than `SimdFFT`, and on the laptop slower for large sizes; going to

512 bits adds a factor of up to three (Table 3). Most of the gain therefore comes from using the full register.

C++26 `std::simd`. On the laptop the standard-only C++26 version is as fast as the one built on `<experimental/simd>`; on the VM it is up to a quarter slower for small sizes. Rewriting the kernel on vectors of `std::complex` (`fft_cx.hpp`) gives the most readable code and halves the twiddle tables, which makes it 5–15 % faster for the largest sizes. For small sizes it is up to 50 % slower, because the current library cannot permute complex vectors, compiles a complex broadcast to scalar stores, and uses a generic complex product with extra shuffles. Working around the first two with a non-portable cast recovers most of the gap (Tables 5–6, 9–10).

Emulated `simd`. With clang and libc++, where the vectors are emulated, `fastfft` becomes several times slower, `SimdFFT` up to twice, and `fastfft` falls behind `SimdFFT`, because its shuffles turn into element loops (Table 4). The gr4 patch therefore enables `fastfft` only with native vector types.

In gr4. A small patch routes gr4's complex power-of-two FFTs to `fastfft` and streamlines the spectrum block. gr4's unit tests pass, and its streaming FFT block becomes 1.7–2.5× faster for floats (Table 8). The spectrum block hardly changes: the FFT takes about 1 % of its time, the rest goes to magnitude, phase and building the output.

6 Discussion

Using the native vector width and fewer, larger passes is enough for a portable FFT to keep up with a hand-tuned library on both hosts, and nothing in the kernel is specific to x86. Choosing radices per size, as FFTW's planner does, would likely help further. C++26 `std::simd` makes the kernel pure standard C++, and vectors of complex numbers would be its natural formulation once permute, broadcast and the complex product are optimised for them. Emulation shows the other side of portability: code written for native vectors can be slower than simpler code when the vectors are emulated, so a library should check which backend it got.

Code and contributions. `https://github.com/dennisklein/fastfft` (code under gr4's licence, text and data CC BY 4.0). D.K. posed the question and steered and reviewed the work; Claude, an AI model by Anthropic, wrote the code, ran the measurements and drafted the text.

AI usage. The work was done in one Claude Code session spanning about 15 hours, including measurement runs, using a single Claude model and no other AI tools. It took 480 model requests with 0.4 million output tokens and 104 million input tokens, 99 % of which were cached re-reads of the growing conversation. The human time was spent on steering, review and the second host's measurements.

References

[1] GNU Radio 4.0, `https://github.com/fair-acc/gnuradio4`, rev. `af62235`.

[2] J. Pommier, PFFFT: a pretty fast FFT, `https://bitbucket.org/jpommier/pffft`.

[3] ISO/IEC TS 19570:2018, Parallelism TS v2 (`std::experimental::simd`).

[4] M. Kretz et al., P1928: `std::simd` for C++26; implementation: GCC trunk `575b8ec6ec`.

[5] M. Kretz, vir-simd, `https://github.com/mattkretz/vir-simd`, v0.4.4.

[6] M. Frigo, S. G. Johnson, The design and implementation of FFTW3, Proc. IEEE 93(2):216–231, 2005.

[7] T. G. Stockham, High-speed convolution and correlation, AFIPS SJCC, 1966.

[8] C. Van Loan, Computational Frameworks for the Fast Fourier Transform, SIAM, 1992.

[9] B. Efron, R. J. Tibshirani, An Introduction to the Bootstrap, Chapman & Hall, 1993.

[10] T. Hoefler, R. Belli, Scientific benchmarking of parallel computing systems, SC'15, 2015.

Appendix: measurement data

Table 1: Complex float, GCC 14.2: median ns per transform [95 % CI] and speedup of fastfft (median of paired ratios [95 % CI]).

N	fastfft	gr4 SimdFFT	FFTW 3.3.10	vs. SimdFFT	vs. FFTW
2^6	35.8 [34.8, 36.7]	117 [116, 122]	45.6 [44.9, 46.5]	3.32 [3.28, 3.37]	1.27 [1.25, 1.29]
2^7	69.8 [69.5, 70.2]	236 [235, 239]	79.6 [78.8, 84.7]	3.39 [3.38, 3.42]	1.14 [1.13, 1.18]
2^8	163 [161, 165]	471 [469, 475]	180 [172, 182]	2.92 [2.89, 2.93]	1.08 [1.05, 1.12]
2^9	315 [311, 318]	1035 [1028, 1051]	361 [353, 365]	3.32 [3.29, 3.36]	1.13 [1.12, 1.15]
2^{10}	658 [650, 663]	2123 [2113, 2142]	769 [755, 804]	3.27 [3.25, 3.30]	1.17 [1.16, 1.20]
2^{11}	1800 [1789, 1813]	4967 [4947, 4999]	1709 [1689, 1723]	2.75 [2.74, 2.76]	0.94 [0.93, 0.95]
2^{12}	4926 [4906, 4952]	12.5 k [12.3 k, 12.9 k]	4245 [4222, 4284]	2.54 [2.52, 2.58]	0.86 [0.85, 0.87]
2^{13}	10.3 k [10.2 k, 10.4 k]	30.1 k [29.9 k, 30.6 k]	12.9 k [12.8 k, 13.1 k]	2.95 [2.93, 2.97]	1.26 [1.24, 1.28]
2^{14}	21.4 k [21.1 k, 21.6 k]	60.9 k [60.4 k, 61.4 k]	28.6 k [28.3 k, 29.1 k]	2.86 [2.84, 2.89]	1.35 [1.33, 1.37]
2^{15}	45.0 k [44.5 k, 45.6 k]	133.0 k [132.7 k, 133.6 k]	61.9 k [61.5 k, 62.6 k]	2.98 [2.96, 3.00]	1.38 [1.36, 1.39]
2^{16}	143.2 k [141.3 k, 145.7 k]	275.8 k [274.0 k, 278.1 k]	146.7 k [141.1 k, 150.2 k]	1.94 [1.92, 1.96]	1.02 [1.01, 1.04]
2^{17}	411.5 k [406.9 k, 416.6 k]	734.3 k [721.2 k, 742.5 k]	394.9 k [389.4 k, 400.1 k]	1.78 [1.76, 1.80]	0.96 [0.94, 0.98]
2^{18}	1.031 M [1.021 M, 1.042 M]	1.835 M [1.814 M, 1.851 M]	898.7 k [886.1 k, 911.2 k]	1.79 [1.78, 1.81]	0.87 [0.86, 0.89]
2^{19}	2.105 M [2.083 M, 2.121 M]	4.429 M [4.378 M, 4.506 M]	4.373 M [4.293 M, 4.423 M]	2.14 [2.11, 2.15]	2.04 [1.99, 2.09]
2^{20}	5.474 M [5.427 M, 5.542 M]	8.954 M [8.908 M, 9.016 M]	8.029 M [7.976 M, 8.078 M]	1.65 [1.64, 1.66]	1.46 [1.46, 1.49]

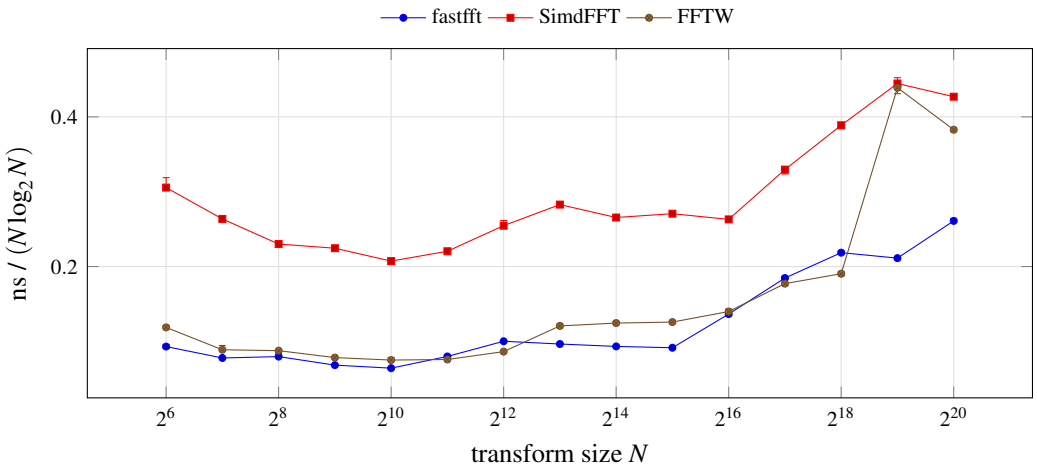


Figure 2: Complex float, GCC 14.2 (Table 1). Time per transform divided by $N \log_2 N$ (lower is faster), error bars: 95 % CI of the median.

Table 2: Complex double, GCC 14.2: median ns per transform [95 % CI] and speedup of fastfft (median of paired ratios [95 % CI]).

N	fastfft	gr4 SimdFFT	FFTW 3.3.10	vs. SimdFFT	vs. FFTW
2^6	74.7 [73.2, 76.2]	132 [131, 135]	84.6 [83.2, 85.4]	1.82 [1.80, 1.83]	1.13 [1.10, 1.15]
2^7	161 [159, 164]	286 [284, 288]	193 [191, 194]	1.79 [1.77, 1.80]	1.20 [1.18, 1.22]
2^8	382 [379, 385]	605 [601, 610]	396 [391, 401]	1.59 [1.58, 1.61]	1.02 [1.01, 1.04]
2^9	837 [827, 851]	1389 [1377, 1409]	884 [873, 902]	1.67 [1.66, 1.69]	1.04 [1.03, 1.05]
2^{10}	2146 [2130, 2163]	3165 [3144, 3181]	1958 [1939, 1975]	1.47 [1.47, 1.48]	0.92 [0.91, 0.92]
2^{11}	4992 [4953, 5017]	9923 [9854, 9970]	4389 [4333, 4520]	1.99 [1.98, 2.01]	0.88 [0.86, 0.89]
2^{12}	11.8 k [11.7 k, 12.0 k]	18.2 k [18.1 k, 18.4 k]	10.9 k [10.8 k, 11.3 k]	1.56 [1.54, 1.58]	0.93 [0.93, 0.94]
2^{13}	24.7 k [24.4 k, 25.0 k]	40.7 k [40.5 k, 41.0 k]	24.7 k [24.4 k, 24.9 k]	1.66 [1.65, 1.68]	1.00 [0.98, 1.02]
2^{14}	52.4 k [52.0 k, 52.9 k]	88.5 k [87.7 k, 89.2 k]	58.2 k [57.3 k, 58.8 k]	1.68 [1.67, 1.69]	1.12 [1.10, 1.14]
2^{15}	136.0 k [134.0 k, 143.3 k]	198.7 k [196.9 k, 204.6 k]	125.6 k [123.8 k, 130.5 k]	1.45 [1.42, 1.46]	0.92 [0.91, 0.93]
2^{16}	405.7 k [400.0 k, 409.7 k]	523.1 k [519.1 k, 528.0 k]	284.3 k [281.4 k, 287.0 k]	1.31 [1.29, 1.33]	0.71 [0.70, 0.71]
2^{17}	999.9 k [991.1 k, 1.008 M]	1.866 M [1.850 M, 1.876 M]	702.1 k [687.6 k, 712.8 k]	1.86 [1.84, 1.87]	0.70 [0.69, 0.71]
2^{18}	2.083 M [2.059 M, 2.100 M]	4.027 M [3.977 M, 4.054 M]	1.486 M [1.471 M, 1.495 M]	1.91 [1.90, 1.94]	0.72 [0.71, 0.73]
2^{19}	4.599 M [4.536 M, 4.679 M]	8.812 M [8.739 M, 8.854 M]	5.187 M [5.157 M, 5.221 M]	1.90 [1.89, 1.93]	1.13 [1.11, 1.14]
2^{20}	12.501 M [12.066 M, 12.788 M]	18.515 M [18.254 M, 18.966 M]	10.232 M [10.180 M, 10.375 M]	1.51 [1.45, 1.57]	0.85 [0.84, 0.87]

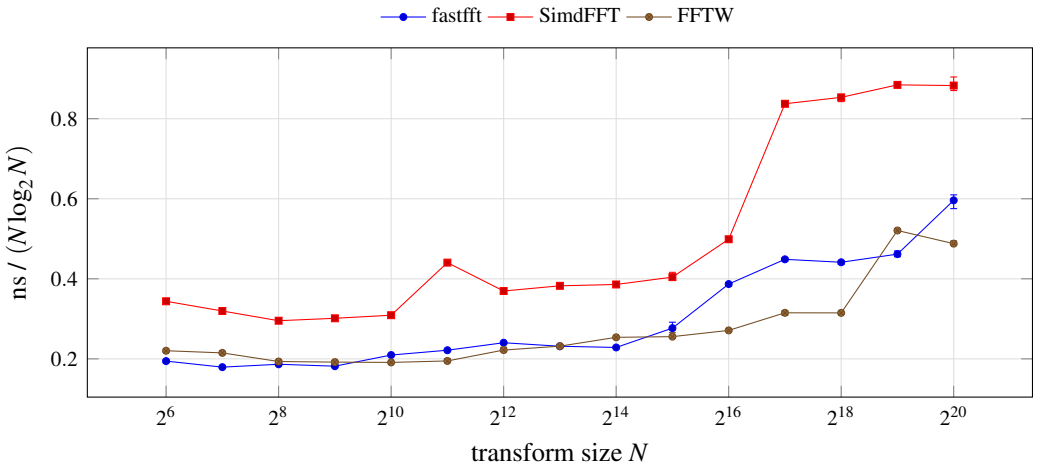


Figure 3: Complex double, GCC 14.2 (Table 2). Time per transform divided by $N \log_2 N$ (lower is faster), error bars: 95 % CI of the median.

Table 3: Vector width, complex float: fastfft compiled for 128, 256, 512-bit vectors (`FASTFFT_SIMD_BYTES`), median ns per transform [95 % CI]; SimdFFT for reference.

N	128 bit	256 bit	512 bit	SimdFFT
2^6	65.2 [64.7, 65.9]	48.2 [48.0, 48.4]	35.8 [34.8, 36.7]	117 [116, 122]
2^7	134 [133, 134]	101 [101, 101]	69.8 [69.5, 70.2]	236 [235, 239]
2^8	337 [334, 342]	222 [220, 223]	163 [161, 165]	471 [469, 475]
2^9	682 [678, 688]	448 [446, 451]	315 [311, 318]	1035 [1028, 1051]
2^{10}	1490 [1483, 1502]	989 [978, 1004]	658 [650, 663]	2123 [2113, 2142]
2^{11}	3300 [3280, 3316]	2272 [2260, 2311]	1800 [1789, 1813]	4967 [4947, 4999]
2^{12}	9133 [8964, 9289]	6146 [6123, 6175]	4926 [4906, 4952]	12.5 k [12.3 k, 12.9 k]
2^{13}	18.9 k [18.7 k, 19.0 k]	13.2 k [13.1 k, 13.3 k]	10.3 k [10.2 k, 10.4 k]	30.1 k [29.9 k, 30.6 k]
2^{14}	41.5 k [39.8 k, 42.2 k]	27.6 k [27.5 k, 27.9 k]	21.4 k [21.1 k, 21.6 k]	60.9 k [60.4 k, 61.4 k]
2^{15}	135.7 k [133.8 k, 140.4 k]	74.6 k [74.0 k, 75.3 k]	45.0 k [44.5 k, 45.6 k]	133.0 k [132.7 k, 133.6 k]
2^{16}	299.8 k [297.0 k, 305.3 k]	184.0 k [183.1 k, 185.3 k]	143.2 k [141.3 k, 145.7 k]	275.8 k [274.0 k, 278.1 k]
2^{17}	641.1 k [639.7 k, 644.8 k]	449.9 k [447.1 k, 453.7 k]	411.5 k [406.9 k, 416.6 k]	734.3 k [721.2 k, 742.5 k]
2^{18}	1.413 M [1.407 M, 1.417 M]	1.070 M [1.066 M, 1.076 M]	1.031 M [1.021 M, 1.042 M]	1.835 M [1.814 M, 1.851 M]
2^{19}	3.826 M [3.769 M, 3.896 M]	2.541 M [2.528 M, 2.552 M]	2.105 M [2.083 M, 2.121 M]	4.429 M [4.378 M, 4.506 M]
2^{20}	8.354 M [8.325 M, 8.396 M]	6.035 M [6.011 M, 6.068 M]	5.474 M [5.427 M, 5.542 M]	8.954 M [8.908 M, 9.016 M]

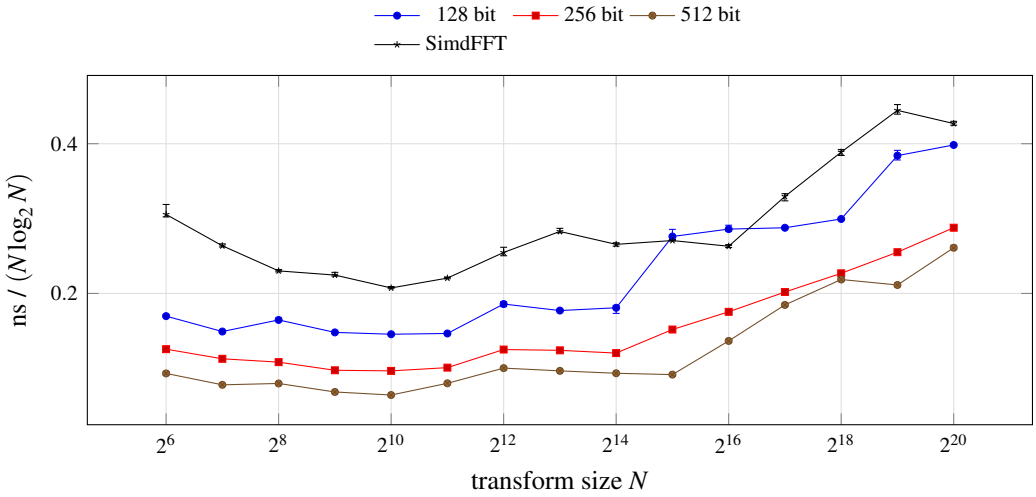


Figure 4: Vector width, complex float (Table 3). Time per transform divided by $N \log_2 N$ (lower is faster), error bars: 95 % CI of the median.

Table 4: Standard library: complex float built with GCC/libstdc++ (native simd) and with clang/libc++ (vir-simd array emulation), median ns per transform [95 % CI].

N	fastfft, libstdc++	fastfft, libc++	SimdFFT, libstdc++	SimdFFT, libc++
2^6	35.8 [34.8, 36.7]	165 [162, 167]	117 [116, 122]	232 [230, 236]
2^7	69.8 [69.5, 70.2]	791 [784, 799]	236 [235, 239]	456 [454, 460]
2^8	163 [161, 165]	1493 [1483, 1524]	471 [469, 475]	889 [882, 903]
2^9	315 [311, 318]	2881 [2843, 2909]	1035 [1028, 1051]	1990 [1975, 2000]
2^{10}	658 [650, 663]	6423 [6379, 6482]	2123 [2113, 2142]	4143 [4119, 4167]
2^{11}	1800 [1789, 1813]	18.2 k [18.1 k, 18.3 k]	4967 [4947, 4999]	9284 [9208, 9353]
2^{12}	4926 [4906, 4952]	37.0 k [36.7 k, 37.1 k]	12.5 k [12.3 k, 12.9 k]	21.0 k [20.9 k, 21.2 k]
2^{13}	10.3 k [10.2 k, 10.4 k]	74.6 k [73.7 k, 76.5 k]	30.1 k [29.9 k, 30.6 k]	46.8 k [46.4 k, 48.1 k]
2^{14}	21.4 k [21.1 k, 21.6 k]	160.7 k [159.8 k, 165.5 k]	60.9 k [60.4 k, 61.4 k]	98.8 k [97.2 k, 105.4 k]
2^{15}	45.0 k [44.5 k, 45.6 k]	401.9 k [399.3 k, 405.2 k]	133.0 k [132.7 k, 133.6 k]	209.7 k [208.0 k, 212.4 k]
2^{16}	143.2 k [141.3 k, 145.7 k]	839.8 k [833.3 k, 856.2 k]	275.8 k [274.0 k, 278.1 k]	443.7 k [438.0 k, 451.4 k]
2^{17}	411.5 k [406.9 k, 416.6 k]	1.756 M [1.736 M, 1.779 M]	734.3 k [721.2 k, 742.5 k]	1.038 M [1.029 M, 1.057 M]
2^{18}	1.031 M [1.021 M, 1.042 M]	3.792 M [3.758 M, 3.837 M]	1.835 M [1.814 M, 1.851 M]	2.379 M [2.359 M, 2.410 M]
2^{19}	2.105 M [2.083 M, 2.121 M]	9.006 M [8.951 M, 9.051 M]	4.429 M [4.378 M, 4.506 M]	5.314 M [5.237 M, 5.347 M]
2^{20}	5.474 M [5.427 M, 5.542 M]	19.933 M [19.450 M, 20.493 M]	8.954 M [8.908 M, 9.016 M]	11.341 M [11.250 M, 11.429 M]

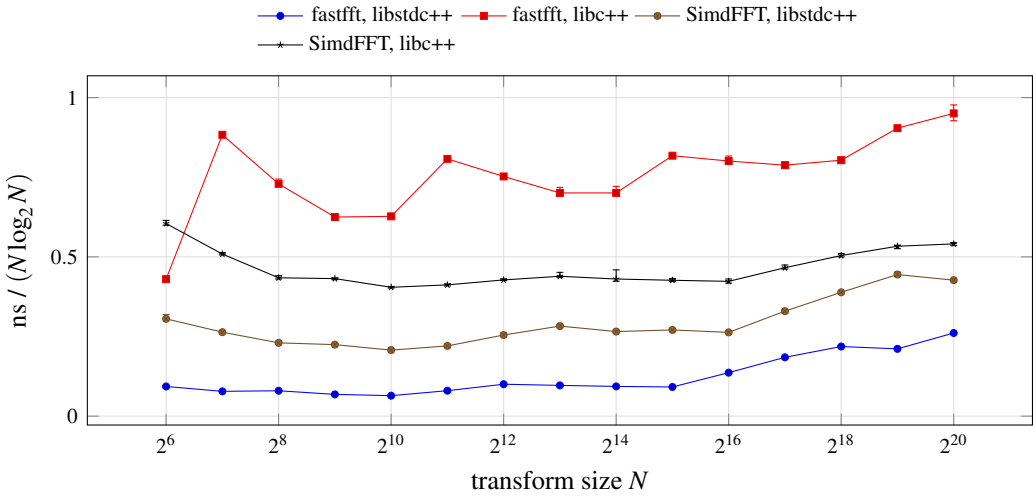


Figure 5: Standard library, complex float (Table 4). Time per transform divided by $N \log_2 N$ (lower is faster), error bars: 95 % CI of the median.

Table 5: GCC trunk (17.0) with `-std=c++26`, complex float: median ns per transform [95 % CI]. fastfft on `<experimental/simd>` (GCC 14 and trunk), on C++26 `std::simd`, and fastfft::cx on `std::simd::vec<std::complex<float>>` (plain, and with zip/broadcast via `bit_cast` to a real vector); SimdFFT built with trunk for reference.

N	exp., GCC 14	exp., trunk	C++26	cx	cx, bit_cast	SimdFFT
2^6	35.8 [34.8, 36.7]	35.3 [35.1, 35.5]	44.0 [43.7, 44.6]	57.8 [57.1, 58.5]	46.7 [46.4, 46.9]	106 [105, 107]
2^7	69.8 [69.5, 70.2]	75.7 [72.1, 86.3]	83.8 [83.5, 84.4]	108 [108, 109]	92.6 [92.1, 93.3]	219 [218, 220]
2^8	163 [161, 165]	173 [167, 176]	182 [181, 184]	240 [239, 242]	198 [197, 200]	445 [442, 447]
2^9	315 [311, 318]	302 [299, 307]	340 [336, 343]	499 [496, 507]	389 [387, 392]	988 [981, 997]
2^{10}	658 [650, 663]	654 [641, 660]	719 [703, 742]	1063 [1053, 1079]	817 [813, 821]	2096 [2072, 2163]
2^{11}	1800 [1789, 1813]	1813 [1799, 1824]	1809 [1805, 1819]	2257 [2245, 2271]	1931 [1907, 1952]	4809 [4782, 4853]
2^{12}	4926 [4906, 4952]	5043 [5000, 5089]	5020 [4996, 5056]	5814 [5777, 5858]	4970 [4932, 4998]	12.1 k [12.1 k, 12.2 k]
2^{13}	10.3 k [10.2 k, 10.4 k]	10.2 k [10.0 k, 10.3 k]	10.7 k [10.5 k, 10.8 k]	12.6 k [12.6 k, 12.7 k]	10.7 k [10.6 k, 11.1 k]	29.3 k [29.2 k, 29.4 k]
2^{14}	21.4 k [21.1 k, 21.6 k]	20.5 k [20.4 k, 20.6 k]	21.4 k [21.1 k, 21.7 k]	25.9 k [25.7 k, 26.3 k]	20.9 k [20.8 k, 21.1 k]	59.0 k [58.6 k, 59.4 k]
2^{15}	45.0 k [44.5 k, 45.6 k]	44.2 k [44.0 k, 44.5 k]	47.7 k [47.4 k, 48.1 k]	54.1 k [53.8 k, 54.8 k]	46.0 k [45.8 k, 46.5 k]	134.2 k [133.5 k, 135.0 k]
2^{16}	143.2 k [141.3 k, 145.7 k]	139.5 k [137.8 k, 143.9 k]	136.9 k [135.6 k, 137.9 k]	129.4 k [128.4 k, 130.4 k]	120.6 k [119.1 k, 122.0 k]	273.4 k [272.4 k, 275.1 k]
2^{17}	411.5 k [406.9 k, 416.6 k]	395.9 k [390.5 k, 398.2 k]	398.9 k [388.3 k, 403.1 k]	353.7 k [349.6 k, 358.1 k]	337.5 k [335.3 k, 340.0 k]	717.0 k [706.2 k, 730.2 k]
2^{18}	1.031 M [1.021 M, 1.042 M]	1.009 M [997.7 k, 1.019 M]	1.005 M [998.9 k, 1.013 M]	883.7 k [880.2 k, 890.0 k]	908.8 k [901.5 k, 915.2 k]	1.805 M [1.794 M, 1.812 M]
2^{19}	2.105 M [2.083 M, 2.121 M]	2.041 M [2.034 M, 2.053 M]	2.147 M [2.120 M, 2.173 M]	1.902 M [1.887 M, 1.920 M]	1.948 M [1.927 M, 1.960 M]	4.531 M [4.470 M, 4.587 M]
2^{20}	5.474 M [5.427 M, 5.542 M]	4.957 M [4.865 M, 5.049 M]	5.837 M [5.736 M, 5.979 M]	5.116 M [5.068 M, 5.231 M]	4.680 M [4.654 M, 4.705 M]	9.211 M [9.058 M, 9.323 M]

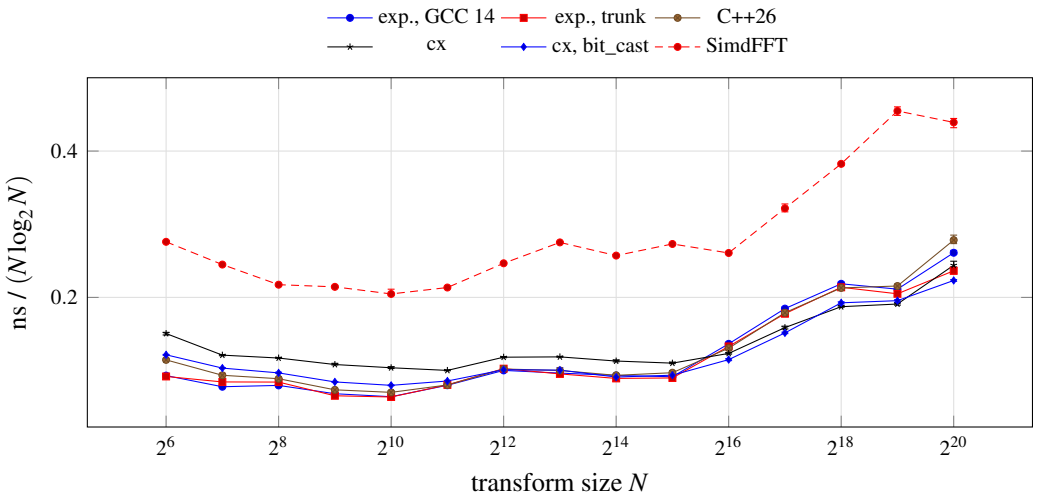


Figure 6: GCC trunk, complex float (Table 5). Time per transform divided by $N \log_2 N$ (lower is faster), error bars: 95 % CI of the median.

Table 6: GCC trunk (17.0) with `-std=c++26`, complex double: median ns per transform [95 % CI]. fastfft on `<experimental/simd>` (GCC 14 and trunk), on C++26 `std::simd`, and fastfft::cx on `std::simd::vec<std::complex<double>>` (plain, and with zip/broadcast via `bit_cast` to a real vector); SimdFFT built with trunk for reference.

N	exp., GCC 14	exp., trunk	C++26	cx	cx, bit_cast	SimdFFT
2^6	74.7	78.6	87.1	108	94.3	132
	[73.2, 76.2]	[75.7, 79.7]	[85.7, 88.3]	[107, 110]	[91.8, 98.8]	[131, 133]
2^7	161	164	179	227	191	284
	[159, 164]	[162, 167]	[176, 181]	[226, 229]	[189, 193]	[281, 286]
2^8	382	372	434	483	455	608
	[379, 385]	[368, 374]	[430, 440]	[479, 488]	[451, 459]	[602, 610]
2^9	837	813	944	1092	954	1444
	[827, 851]	[807, 822]	[934, 954]	[1078, 1107]	[940, 967]	[1414, 1485]
2^{10}	2146	2185	2206	2282	2114	3251
	[2130, 2163]	[2166, 2209]	[2195, 2226]	[2274, 2310]	[2099, 2129]	[3237, 3288]
2^{11}	4992	5105	4962	4984	4939	10.1 k
	[4953, 5017]	[5047, 5165]	[4907, 5009]	[4948, 5030]	[4870, 4994]	[10.0 k, 10.1 k]
2^{12}	11.8 k	11.5 k	12.0 k	12.7 k	11.8 k	18.0 k
	[11.7 k, 12.0 k]	[11.4 k, 11.6 k]	[11.9 k, 12.0 k]	[12.6 k, 12.8 k]	[11.6 k, 12.2 k]	[18.0 k, 18.1 k]
2^{13}	24.7 k	23.8 k	24.4 k	26.1 k	23.6 k	39.9 k
	[24.4 k, 25.0 k]	[23.6 k, 24.0 k]	[24.3 k, 24.6 k]	[25.8 k, 26.2 k]	[23.4 k, 23.9 k]	[39.8 k, 40.1 k]
2^{14}	52.4 k	49.6 k	52.9 k	54.1 k	49.6 k	87.6 k
	[52.0 k, 52.9 k]	[49.4 k, 49.9 k]	[52.2 k, 53.4 k]	[53.7 k, 54.4 k]	[49.4 k, 49.7 k]	[87.3 k, 88.0 k]
2^{15}	136.0 k	135.9 k	133.4 k	117.0 k	114.4 k	200.5 k
	[134.0 k, 143.3 k]	[133.7 k, 137.7 k]	[132.1 k, 135.2 k]	[116.6 k, 117.9 k]	[112.4 k, 115.2 k]	[199.5 k, 201.4 k]
2^{16}	405.7 k	399.7 k	386.0 k	354.8 k	344.9 k	540.2 k
	[400.0 k, 409.7 k]	[394.2 k, 404.0 k]	[382.6 k, 402.1 k]	[350.5 k, 361.5 k]	[343.0 k, 347.1 k]	[532.0 k, 549.1 k]
2^{17}	999.9 k	1.008 M	1.015 M	899.3 k	880.9 k	1.860 M
	[991.1 k, 1.008 M]	[998.0 k, 1.019 M]	[1.005 M, 1.027 M]	[891.9 k, 905.9 k]	[874.2 k, 885.1 k]	[1.836 M, 1.881 M]
2^{18}	2.083 M	2.103 M	2.182 M	2.005 M	1.928 M	4.134 M
	[2.059 M, 2.100 M]	[2.070 M, 2.139 M]	[2.149 M, 2.200 M]	[1.983 M, 2.027 M]	[1.912 M, 1.947 M]	[4.089 M, 4.175 M]
2^{19}	4.599 M	4.199 M	5.022 M	4.422 M	3.824 M	8.947 M
	[4.536 M, 4.679 M]	[4.173 M, 4.236 M]	[4.920 M, 5.213 M]	[4.350 M, 4.528 M]	[3.791 M, 3.861 M]	[8.887 M, 9.026 M]
2^{20}	12.501 M	10.151 M	13.141 M	11.766 M	9.922 M	19.523 M
	[12.066 M, 12.788 M]	[10.061 M, 10.241 M]	[12.723 M, 13.383 M]	[11.528 M, 11.989 M]	[9.834 M, 10.069 M]	[18.817 M, 19.845 M]

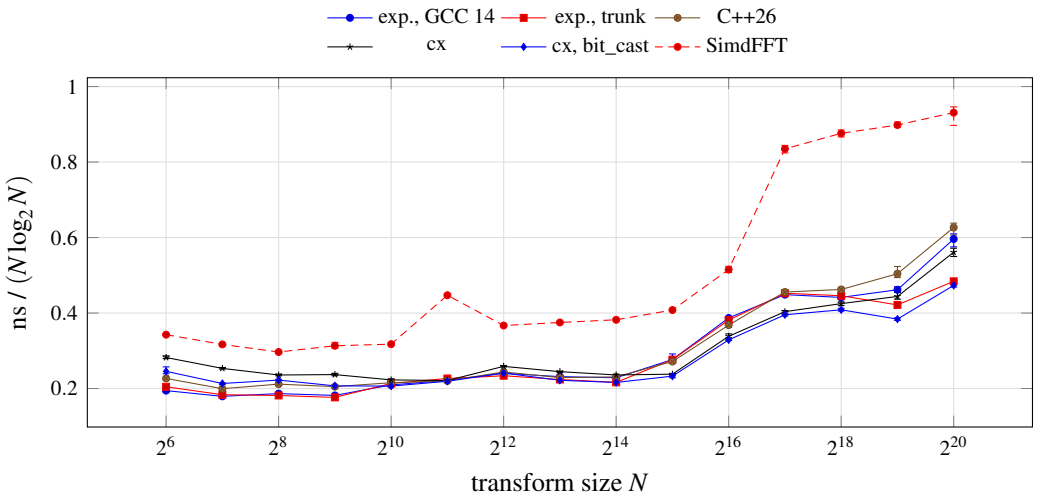


Figure 7: GCC trunk, complex double (Table 6). Time per transform divided by $N \log_2 N$ (lower is faster), error bars: 95 % CI of the median.

Table 7: Accuracy: largest $\max_k |X_k - \hat{X}_k| / \max_k |\hat{X}_k|$ over all N, against a long-double reference.

	fastfft	SimdFFT	FFTW
float	2.1×10^{-7}	4.8×10^{-7}	2.1×10^{-7}
double	6.0×10^{-16}	8.4×10^{-16}	3.3×10^{-16}

Table 8: gnuradio4 blocks before and after the patch, gnuradio4's benchmark facility: median call time over runs, speedup as median of paired per-run ratios [95 % CI].

benchmark row	unpatched	patched	speedup
`std::complex<float> - fft`	70.5 µs	68.9 µs	1.06 [0.98, 1.07]
`std::complex<float> - fftCT`	79.7 µs	80.6 µs	1.00 [0.95, 1.01]
`std::complex<double> - fft`	93.0 µs	60.2 µs	1.55 [1.45, 1.59]
`std::complex<double> - fftCT`	98.0 µs	97.9 µs	0.99 [0.91, 1.21]
`float - fft`	23.3 µs	22.7 µs	1.00 [0.97, 1.07]
`double - fft`	27.4 µs	27.7 µs	1.01 [0.81, 1.10]
`FFT<float32> stream N= 64`	138.3 µs	53.5 µs	2.55 [2.53, 4.34]
`FFT<float32> stream N= 256`	135.3 µs	56.6 µs	2.39 [2.33, 4.00]
`FFT<float32> stream N= 1024`	150.4 µs	59.0 µs	2.52 [2.49, 2.57]
`FFT<float32> stream N= 4096`	210.3 µs	97.6 µs	2.14 [2.08, 2.18]
`FFT<float32> stream N= 16384`	256.6 µs	103.9 µs	2.48 [2.37, 2.60]
`FFT<float32> stream N= 65536`	300.1 µs	178.0 µs	1.70 [1.58, 1.73]
`FFT<float32> stream N= 262144`	1979.0 µs	1159.0 µs	1.76 [1.64, 1.83]
`FFT<float32> stream N=1048576`	9300.0 µs	5582.0 µs	1.66 [1.65, 1.73]
`FFT<float64> stream N= 64`	195.0 µs	144.3 µs	1.34 [1.19, 1.43]
`FFT<float64> stream N= 256`	228.3 µs	161.8 µs	1.42 [1.32, 1.47]
`FFT<float64> stream N= 1024`	274.7 µs	205.5 µs	1.34 [1.16, 1.37]
`FFT<float64> stream N= 4096`	367.3 µs	276.4 µs	1.33 [1.26, 1.41]
`FFT<float64> stream N= 16384`	444.5 µs	315.7 µs	1.43 [1.35, 1.60]
`FFT<float64> stream N= 65536`	616.6 µs	490.2 µs	1.33 [1.16, 1.55]
`FFT<float64> stream N= 262144`	4268.0 µs	2326.0 µs	1.84 [1.74, 1.87]
`FFT<float64> stream N=1048576`	19490.0 µs	11470.0 µs	1.74 [1.66, 1.76]

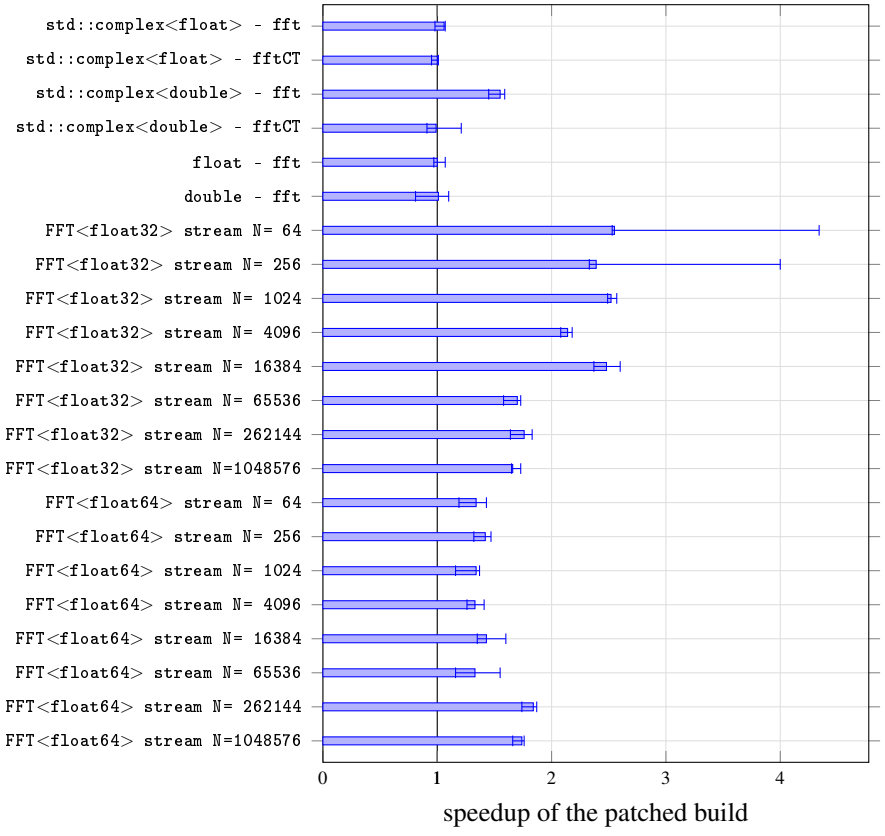


Figure 8: gnuradio4 blocks, speedup of the patched build (Table 8), error bars: 95 % CI of the median of paired ratios.

Table 9: Second host, AMD Ryzen AI 9 HX 370 (one core, GCC 16.2.1), complex float: median ns per transform [95 % CI], speedups of fastfft (paired), and the GCC trunk builds (`-std=c++26`): fastfft on C++26 `std::simd`, on `<experimental/simd>`, and fastfft::cx.

N	fastfft	SimdFFT	FFTW	vs. SimdFFT	vs. FFTW	C++26	exp., trunk	cx
2^{6}	23.6	52.0	21.8	2.21	0.91	26.6	27.2	32.0
	[23.6, 23.6]	[52.0, 52.1]	[21.7, 21.8]	[2.20, 2.22]	[0.91, 0.92]	[26.5, 26.7]	[27.2, 27.3]	[32.0, 32.1]
2^{7}	44.0	117	38.5	2.68	0.87	45.3	45.0	58.3
	[43.9, 44.0]	[117, 117]	[38.5, 38.7]	[2.67, 2.69]	[0.87, 0.88]	[45.2, 45.4]	[44.9, 45.1]	[58.1, 58.4]
2^{8}	99.7	222	78.3	2.22	0.79	100	99.3	132
	[99.5, 99.9]	[222, 222]	[78.0, 78.6]	[2.22, 2.23]	[0.79, 0.79]	[100, 100]	[99.3, 99.4]	[132, 132]
2^{9}	189	504	172	2.67	0.91	189	192	274
	[188, 190]	[504, 505]	[171, 173]	[2.65, 2.68]	[0.91, 0.92]	[189, 189]	[192, 193]	[273, 274]
2^{10}	399	1029	428	2.59	1.07	398	390	583
	[397, 400]	[1027, 1030]	[424, 431]	[2.58, 2.59]	[1.07, 1.08]	[397, 399]	[390, 393]	[583, 584]
2^{11}	878	2248	938	2.56	1.06	872	867	1264
	[875, 881]	[2246, 2249]	[937, 941]	[2.55, 2.57]	[1.06, 1.06]	[871, 873]	[866, 871]	[1261, 1267]
2^{12}	1994	4769	2229	2.39	1.11	1996	1990	2873
	[1991, 1998]	[4760, 4789]	[2218, 2238]	[2.39, 2.40]	[1.10, 1.11]	[1993, 2001]	[1986, 2000]	[2866, 2877]
2^{13}	4489	10.5 k	5991	2.33	1.35	4500	4517	6235
	[4476, 4510]	[10.5 k, 10.5 k]	[5982, 6014]	[2.32, 2.34]	[1.34, 1.35]	[4488, 4529]	[4510, 4522]	[6215, 6250]
2^{14}	9603	21.1 k	15.0 k	2.19	1.60	9733	9393	13.0 k
	[9575, 9622]	[21.0 k, 21.1 k]	[15.0 k, 15.1 k]	[2.19, 2.20]	[1.59, 1.62]	[9711, 9773]	[9381, 9408]	[13.0 k, 13.0 k]
2^{15}	26.0 k	46.6 k	34.5 k	1.82	1.33	25.0 k	26.5 k	31.0 k
	[25.8 k, 26.1 k]	[46.5 k, 46.8 k]	[34.1 k, 34.6 k]	[1.80, 1.83]	[1.32, 1.34]	[24.9 k, 25.2 k]	[26.5 k, 26.6 k]	[31.0 k, 31.3 k]
2^{16}	65.3 k	100.6 k	74.7 k	1.54	1.16	63.3 k	58.9 k	71.4 k
	[65.0 k, 65.8 k]	[100.0 k, 100.9 k]	[74.3 k, 75.1 k]	[1.53, 1.55]	[1.15, 1.17]	[63.1 k, 63.4 k]	[58.7 k, 59.1 k]	[71.3 k, 71.5 k]
2^{17}	149.8 k	244.3 k	189.9 k	1.63	1.26	150.3 k	127.8 k	160.8 k
	[149.0 k, 150.5 k]	[243.7 k, 244.7 k]	[189.4 k, 190.7 k]	[1.62, 1.64]	[1.26, 1.29]	[149.8 k, 150.8 k]	[127.4 k, 128.0 k]	[159.9 k, 161.4 k]
2^{18}	299.8 k	452.3 k	418.3 k	1.52	1.41	309.0 k	277.8 k	314.5 k
	[294.6 k, 301.9 k]	[451.4 k, 454.3 k]	[416.4 k, 421.0 k]	[1.50, 1.54]	[1.39, 1.43]	[305.0 k, 312.5 k]	[275.1 k, 279.4 k]	[312.8 k, 318.4 k]
2^{19}	905.3 k	1.039 M	2.140 M	1.15	2.36	911.6 k	878.9 k	858.6 k
	[901.0 k, 908.7 k]	[1.036 M, 1.041 M]	[2.131 M, 2.151 M]	[1.14, 1.16]	[2.34, 2.38]	[909.6 k, 915.1 k]	[870.2 k, 886.0 k]	[855.7 k, 861.5 k]
2^{20}	2.628 M	4.592 M	4.816 M	1.75	1.84	2.679 M	2.495 M	2.541 M
	[2.584 M, 2.658 M]	[4.558 M, 4.639 M]	[4.789 M, 4.842 M]	[1.71, 1.78]	[1.81, 1.85]	[2.656 M, 2.691 M]	[2.489 M, 2.513 M]	[2.531 M, 2.551 M]

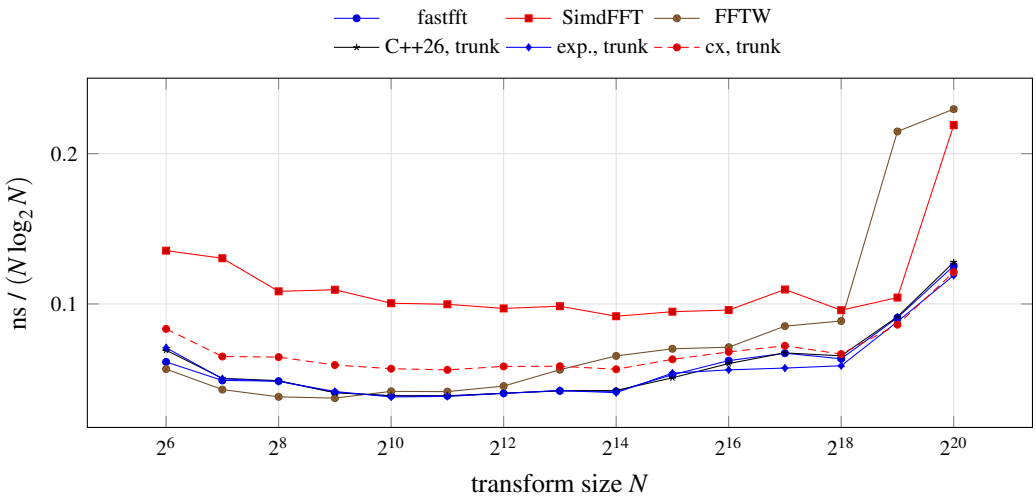


Figure 9: Second host, complex float (Table 9). Time per transform divided by $N \log_2 N$ (lower is faster), error bars: 95 % CI of the median.

Table 10: Second host, AMD Ryzen AI 9 HX 370 (one core, GCC 16.2.1), complex double: median ns per transform [95 % CI], speedups of fastfft (paired), and the GCC trunk builds (`-std=c++26`): fastfft on C++26 `std::simd`, on `<experimental/simd>`, and fastfft::cx.

N	fastfft	SimdFFT	FFTW	vs. SimdFFT	vs. FFTW	C++26	exp., trunk	cx
2^{6}	40.4	62.3	37.4	1.54	0.93	40.2	39.7	54.8
	[40.3, 40.4]	[62.2, 62.3]	[37.3, 37.6]	[1.54, 1.54]	[0.92, 0.93]	[40.1, 40.4]	[39.7, 39.8]	[54.7, 54.9]
2^{7}	82.4	128	70.8	1.56	0.86	80.4	81.2	113
	[82.3, 82.6]	[128, 129]	[70.7, 70.9]	[1.55, 1.56]	[0.86, 0.86]	[80.1, 80.5]	[81.1, 81.2]	[113, 113]
2^{8}	207	250	157	1.22	0.76	203	203	272
	[206, 207]	[249, 250]	[157, 158]	[1.21, 1.22]	[0.76, 0.76]	[203, 204]	[203, 204]	[271, 272]
2^{9}	414	584	369	1.42	0.89	412	412	573
	[413, 415]	[583, 586]	[368, 371]	[1.41, 1.42]	[0.89, 0.89]	[412, 413]	[411, 413]	[572, 573]
2^{10}	926	1215	901	1.32	0.98	912	921	1189
	[925, 927]	[1213, 1219]	[900, 904]	[1.32, 1.32]	[0.98, 0.99]	[909, 919]	[920, 922]	[1184, 1193]
2^{11}	1991	3138	1810	1.58	0.91	1981	1960	2520
	[1985, 2000]	[3131, 3151]	[1798, 1820]	[1.58, 1.58]	[0.90, 0.91]	[1977, 1984]	[1958, 1961]	[2516, 2523]
2^{12}	5028	6975	4923	1.39	1.04	4913	5007	6107
	[5000, 5063]	[6949, 6988]	[4911, 4934]	[1.39, 1.39]	[1.04, 1.04]	[4875, 4946]	[5004, 5014]	[6091, 6126]
2^{13}	10.6 k	15.0 k	10.9 k	1.42	1.03	10.1 k	10.6 k	12.7 k
	[10.6 k, 10.6 k]	[14.9 k, 15.0 k]	[10.8 k, 10.9 k]	[1.41, 1.42]	[1.02, 1.03]	[10.1 k, 10.1 k]	[10.6 k, 10.6 k]	[12.7 k, 12.8 k]
2^{14}	20.7 k	31.1 k	25.7 k	1.51	1.25	22.4 k	22.6 k	26.9 k
	[20.6 k, 20.7 k]	[31.0 k, 31.2 k]	[25.6 k, 25.8 k]	[1.50, 1.51]	[1.24, 1.25]	[22.3 k, 22.6 k]	[22.5 k, 22.6 k]	[26.8 k, 27.3 k]
2^{15}	53.7 k	75.8 k	57.5 k	1.38	1.07	56.0 k	60.3 k	63.3 k
	[53.5 k, 56.1 k]	[75.1 k, 76.8 k]	[57.2 k, 57.8 k]	[1.37, 1.39]	[1.07, 1.07]	[55.9 k, 56.2 k]	[60.2 k, 60.4 k]	[63.2 k, 63.5 k]
2^{16}	146.4 k	184.5 k	144.2 k	1.24	1.02	143.7 k	135.4 k	154.1 k
	[143.9 k, 149.8 k]	[183.3 k, 185.2 k]	[142.0 k, 145.3 k]	[1.23, 1.27]	[1.01, 1.03]	[140.6 k, 146.8 k]	[134.5 k, 135.7 k]	[152.1 k, 159.8 k]
2^{17}	294.0 k	381.6 k	306.2 k	1.31	1.03	299.4 k	289.8 k	319.0 k
	[291.6 k, 296.0 k]	[380.7 k, 383.5 k]	[305.4 k, 306.9 k]	[1.29, 1.32]	[1.03, 1.04]	[298.0 k, 300.3 k]	[288.9 k, 290.6 k]	[317.6 k, 319.9 k]
2^{18}	778.0 k	805.8 k	724.6 k	1.03	0.93	769.7 k	733.1 k	765.5 k
	[772.5 k, 784.7 k]	[801.4 k, 809.3 k]	[722.1 k, 727.9 k]	[1.02, 1.04]	[0.92, 0.94]	[764.3 k, 778.2 k]	[730.9 k, 738.8 k]	[762.7 k, 769.6 k]
2^{19}	2.174 M	3.785 M	3.157 M	1.76	1.47	2.241 M	2.592 M	2.036 M
	[2.120 M, 2.192 M]	[3.748 M, 3.856 M]	[3.139 M, 3.176 M]	[1.72, 1.79]	[1.44, 1.48]	[2.229 M, 2.254 M]	[2.581 M, 2.611 M]	[2.030 M, 2.044 M]
2^{20}	6.631 M	9.960 M	6.958 M	1.50	1.05	6.775 M	6.629 M	5.798 M
	[6.619 M, 6.637 M]	[9.932 M, 9.989 M]	[6.902 M, 7.025 M]	[1.50, 1.51]	[1.05, 1.06]	[6.767 M, 6.800 M]	[6.593 M, 6.654 M]	[5.771 M, 5.862 M]

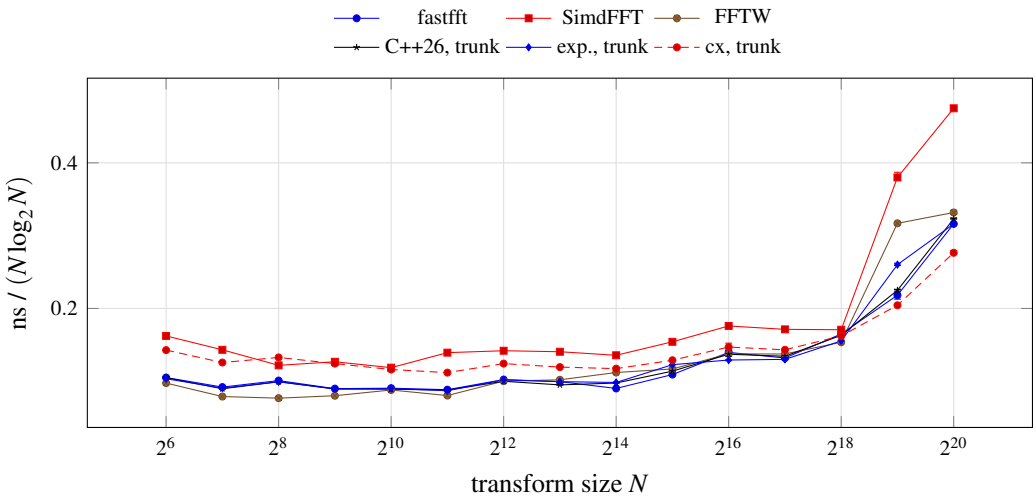


Figure 10: Second host, complex double (Table 10). Time per transform divided by $N \log_2 N$ (lower is faster), error bars: 95 % CI of the median.